

RELAZIONE DI PROGETTO: ALLERGY ALERT

1. Scopo del Progetto

Allergy Alert è un'applicazione mobile sviluppata in **Flutter** per supportare le persone affette da allergie respiratorie o stagionali nella gestione della propria salute quotidiana.

Problema e Soluzione

Le persone allergiche necessitano di monitorare costantemente i fattori ambientali per prevenire l'insorgenza di sintomi acuti. **Allergy Alert** unisce l'analisi in tempo reale dei dati atmosferici con un diario personale dei sintomi, permettendo all'utente di pianificare attività all'aperto, adottare cure preventive e valutare l'efficacia dei propri trattamenti.

Funzionalità Principali

- **Previsioni Pollini e Allergeni (3 Giorni):** Consultazione dei livelli di concentrazione e del rischio allergico per la giornata odierna e per i due giorni successivi, impostati di default sulla città di Roma.
- **Inserimento Sintomi e Gravità:** Schermata dedicata per la registrazione dei sintomi avvertiti, con selezione obbligatoria del livello di gravità e possibilità di aggiungere note o rilievi testuali opzionali.
- **Cronologia del Diario Sintomi:** Visualizzazione ordinata in sequenza cronologica (con i dati più recenti in alto e i più vecchi in basso) comprensiva di data, ora, gravità e nota testuale.
- **Monitoraggio e Prevenzione:** Storico visivo per tracciare l'andamento delle proprie allergie nel tempo, consentendo di prendere precauzioni o avviare cure preventive prima dei picchi allergici.

2. Descrizione della Struttura dei Sorgenti

L'applicazione è sviluppata in **Flutter** e segue il pattern architetturale **MVVM (Model-View-ViewModel)** per garantire una netta separazione tra la struttura dei dati, la logica applicativa e le interfacce grafiche.

Tabella dei Pacchetti Utilizzati (Dependencies)

Dependency	Scopo e Utilizzo
http	Chiamate REST asincrone verso Open-Meteo API
provider	Gestione dello stato e architettura MVVM reattiva

Dependency	Scopo e Utilizzo
shared_preferences	Salvataggio e persistenza locale dei sintomi del diario
Flutter_test	Suite ufficiale di test di unità e di componenti UI per Flutter

Struttura dell'Albero di Progetto (lib/)

```

allergy_alert/
├── lib/
│   ├── models/
│   │   ├── pollen_data.dart      # Modello dati per le previsioni dei pollini
│   │   └── symptom_log.dart      # Modello dati per le registrazioni dei sintomi
│   ├── services/
│   │   ├── api_service.dart      # Gestione dell'integrazione e chiamate all'API di Open-Meteo
│   │   └── storage_service.dart  # Servizio per la persistenza e salvataggio locale dei sintomi
│   ├── viewmodels/
│   │   └── allergy_viewmodel.dart # Gestione dello stato globale e della logica di business
│   ├── views/
│   │   ├── home_screen.dart      # UI principale: previsioni 3 giorni, lista sintomi e banner alert
│   │   └── log_symptom_screen.dart # UI di inserimento: form per gravità e note
│   └── main.dart                 # Entry point dell'app e inizializzazione dell'architettura
└── test/
    ├── viewmodels/
    │   └── allergy_viewmodel_test.dart # Test di unità per la logica di business
    └── views/
        └── home_screen_test.dart      # Test di interfaccia (Widget Test) per la HomeScreen

```

Ruolo dei Moduli e File del Progetto

1. Models (/models):

- **pollen_data.dart:** Definisce la struttura dati dei pollini e dei livelli di concentrazione restituiti dall'API per la finestra temporale dei 3 giorni.
- **symptom_log.dart:** Definisce il modello dell'entità sintomo, memorizzando data, ora, livello di gravità (obbligatorio) ed eventuale nota testuale.

2. Services (/services):

- **api_service.dart:** Si occupa di effettuare le chiamate di rete verso l'API di Open-Meteo per recuperare i dati dei pollini (con coordinate impostate di default su Roma).
- **storage_service.dart:** Gestisce la lettura e la scrittura dei sintomi su memoria locale, garantendo la persistenza dei dati sul dispositivo.

3. ViewModels (/viewmodels):

- **allergy_viewmodel.dart:** Rappresenta il cuore della logica applicativa. Coordina il recupero dati da `api_service.dart` e `storage_service.dart`, gestisce lo stato dell'app e notifica le schermate (Views) quando vi sono nuovi aggiornamenti.

4. Views (/views):

- **home_screen.dart:** Interfaccia grafica principale che mostra sia la card/sezione delle previsioni pollini (oggi e prossimi due giorni) con eventuale banner di alert, sia il diario dei sintomi registrati ordinati dal più recente al più vecchio.
- **log_symptom_screen.dart:** Schermata di form per l'inserimento di un nuovo sintomo, dove l'utente seleziona la gravità obbligatoria e l'eventuale nota prima di effettuare il salvataggio.

- 5. **Main (main.dart):** Punto d'ingresso dell'applicazione che si occupa dell'avvio e della configurazione dell'architettura reattiva.

3. Testing e Validazione

La fase di testing e analisi statica è stata fondamentale per garantire la robustezza del codice, prevenire bug di regressione e verificare il corretto comportamento sia della logica applicativa che dell'interfaccia utente.

3.1 Strategia di testing

In linea con le buone pratiche di sviluppo in Flutter, sono stati realizzati due livelli di test automatizzati:

1. **Unit Testing (Test di Unità):** Diretti a verificare il corretto funzionamento delle classi di logica di business e gestione dello stato (`AllergyViewModel`), garantendo che il recupero dei dati, il calcolo dei livelli di rischio e il salvataggio dei sintomi avvengano come previsto.
2. **Widget Testing (Test di Interfaccia):** Focalizzati sulla verifica del rendering grafico della schermata principale (`HomeScreen`), accertando la corretta presenza degli elementi visivi e la risposta agli eventi di navigazione dell'utente.

3.2 Test di Unità (ViewModel e Logica)

La suite di unit test (`allergy_viewmodel_test.dart`) ha validato lo stato applicativo attraverso diversi scenari:

- **Stato Iniziale e Caricamento:** Verificato che all'avvio dell'app lo stato di `isLoading` sia correttamente gestito e che le liste dei pollini e dei sintomi vengano caricate senza errori.

- **Gestione Connettività e Fallback Offline:** Testata la risposta del sistema in caso di assenza di connessione internet, garantendo che i dati locali rimangano fruibili e che l'utente sia avvisato da un banner dedicato.
- **Inserimento e Persistenza:** Verificato che l'invocazione del metodo `addSymptomLog()` aggiorni correttamente la lista cronologica dei sintomi e notifichi tempestivamente l'interfaccia utente.

3.3 Widget Testing (UI e Navigazione)

Il file `home_screen_test.dart` ha utilizzato la tecnica del *Mocking* (tramite la classe `FakeAllergyViewModel`) per testare in modo isolato la `HomeScreen`:

- **Renderizzazione dei Componenti:** Verificato che la schermata mostri correttamente l'`AppBar` con il titolo "Allergy Alert", la sezione relativa al Diario e il pulsante d'azione rapida `FloatingActionButton`.
- **Test di Navigazione:** Simulato l'evento di tap sul pulsante + per verificare, mediante `tester.pumpAndSettle()`, l'effettiva apertura e il caricamento della schermata di inserimento sintomo (`LogSymptomScreen`).

3.4 Analisi Statica del Codice (Quality Assurance)

Oltre ai test di esecuzione, l'intera codebase è stata verificata tramite il linter ufficiale di Dart (`flutter analyze`). Tutti gli avvisi relativi alle costanti mancanti (`const`), all'immutabilità dei widget e alla dichiarazione delle variabili locali (`prefer_final_locals`) sono stati completamente risolti, ottenendo il risultato `No issues found!`.

4. Punti di Forza

- **Architettura MVVM rigorosa e pulita:**
 - La separazione tra i Modelli (`pollen_data`, `symptom_log`), i Servizi di I/O, il `ViewModel` e le Schermate (`home_screen`, `log_symptom_screen`) rende il codice estremamente ordinato, manutenibile e facile da estendere.
- **Piattaforma Reattiva e Gestione dello Stato Centralizzata:**
 - L'uso di `allergy_viewmodel.dart` come punto unico di gestione dello stato garantisce che l'aggiunta di un sintomo da `log_symptom_screen.dart` si rifletta immediatamente nella lista cronologica di `home_screen.dart` senza incoerenze nei dati.
- **Ordinamento e Usabilità del Diario (User-Centric):**
 - L'ordinamento cronologico decrescente (i dati recenti in alto) permette all'utente di verificare subito il proprio stato di salute attuale, associando in

modo chiaro data, ora e gravità ai dati meteo visualizzati nella stessa schermata.

- **Disaccoppiamento dei Servizi di I/O:**

- L'isolamento di `api_service.dart` e `storage_service.dart` consente di modificare le sorgenti dati (es. passare a un altro provider API o a un database SQLite) senza dover riscrivere le schermate grafiche.

5. Possibili Migliorie

- **Ampliamento dei Dati e Parametri Ambientali (Inquinanti e Nuovi Pollini):**

- **Espansione allergeni:** Estendere la copertura a un numero maggiore di famiglie di pollini (es. parietaria, cupressacee) e spore fungine (es. alternaria).
- **Qualità dell'aria e inquinanti:** Integrare il monitoraggio di parametri ambientali critici per la respirazione come PM10, PM2.5, CO₂ e biossido di azoto (NO₂), poiché l'inquinamento atmosferico accentua spesso le reazioni allergiche.

- **Notifiche Intelligenti e Reminder per la Registrazione dei Sintomi:**

- **Avviso di superamento soglia:** Inviare una notifica push quando i dati meteo indicano che la concentrazione di pollini supera una soglia definita come "grave".
- **Reminder di tracciamento:** Sfruttare la notifica di allerta per ricordare contestualmente all'utente di registrare i propri sintomi nel diario, garantendo una raccolta dati più costante ed efficace proprio nelle giornate a maggior rischio.

- **Geolocalizzazione Dinamica e Selezione Manuale della Posizione:**

- **GPS Automatico:** Rilevamento della posizione dell'utente tramite geolocalizzazione per superare l'impostazione di default su Roma e mostrare i dati dei pollini della posizione corrente.
- **Ricerca Manuale:** Possibilità di cercare e selezionare città diverse per consultare le previsioni quando si pianificano spostamenti.

- **Grafici Statistici di Correlazione:**

- Inserimento di una vista con grafici temporali per sovrapporre la curva della gravità dei sintomi registrati nel diario con l'andamento della concentrazione dei pollini e degli inquinanti.

- **Esportazione del Diario in PDF:**

- Funzionalità per esportare lo storico del diario sintomi in formato PDF per consentire all'utente di mostrarlo al proprio allergologo o medico curante.